Real-Time Multi-Object Tracking (MOT) & Crowd Analytics

Objective

Design and build a robust, production-grade video processing pipeline capable of detecting, tracking, and counting individuals in crowded environments (e.g., transit hubs, retail environments, metro platforms). The system must utilize **YOLO** for object detection and **BoT-SORT** (or ByteTrack) for multi-object tracking to maintain stable identity persistence, output precise bounding boxes, and dynamically calculate the cumulative count of unique individuals observed over time.

Core Functional Requirements

1. Object Detection & Multi-Object Tracking (MOT)

- **Detection:** Leverage a pre-trained or fine-tuned YOLO variant (e.g., YOLOv8, YOLOv9, or YOLOv10/v11) optimized for person detection.
- **Tracking:** Integrate **BoT-SORT** to handle camera motion, temporal occlusions, and state estimation (Kalman filtering + appearance embeddings).
- **Identity Persistence:** The system must accurately assign a persistent, unique integer ID to each person. If a person is briefly occluded by a pillar or another individual, the system should ideally re-identify them with the same ID upon reappearance rather than assigning a new one.

2. Live Analytics & Annotations

- **Visual Overlay:** Render real-time bounding boxes around every detected person with their assigned Track ID clearly labeled (e.g., `Person #42`).
- **Unique Visitor Counter:** Maintain an absolute, cumulative counter of *unique* individuals detected throughout the entire video sequence. Display this live metric prominently in the upper corner of the video frame (e.g., `Total Unique People: 124`).
- **Performance Metrics (Optional but Preferred):** Overlay processing metrics like frames per second (FPS) and inference latency on screen.

3. Execution Interface

- Provide a robust Command Line Interface (CLI) or a minimal API/UI that accepts an arbitrary input video path (`--input video.mp4`) and generates an annotated output video file (`--output result.mp4`) along with a structured analytics log (JSON/CSV) capturing frame-by-frame ID timestamps.

Technical Guidelines

- **Primary Stack:** Python 3.10+, PyTorch, OpenCV (`cv2`).
- **Frameworks:** Ultralytics YOLO framework or native PyTorch deployment.
- **Hardware Acceleration:** The pipeline must automatically detect and utilize CUDA/MPS if available, with a graceful fallback to CPU processing.

- **Zero Hardcoding:** Input dimensions, confidence thresholds, IoU parameters, and tracking configurations (e.g., track buffer, proximity thresholds) must be highly configurable via a configuration file (`config.yaml`) or CLI arguments.

Deliverables

- **Source Code:** Hosted on a public or private Git repository containing highly modular, object-oriented code.
- **Documentation (README.md):**
    - Step-by-step instructions on environment setup, dependency installation (`requirements.txt` or `pyproject.toml`), and local execution.
    - A pipeline architecture diagram illustrating the flow of frames from ingestion, through detection and association matrices, to the rendering and logging layers.
    - A summary of architectural trade-offs made (e.g., choosing a specific YOLO model size like `yolov8n` vs `yolov8m` based on accuracy vs velocity trade-offs).

Expectations

Your pipeline will be evaluated on its architectural cleanliness, algorithmic resilience against dense crowds, and execution efficiency. The interviewer will test your prototype using a **congested test video** (e.g., a bustling metro platform during rush hour) featuring heavy occlusions, crisscrossing pedestrian paths, and variable lighting.